

Ontology

Creation data: 24/05/2024

Update data: 04/02/2026 (include references ontologies)

Document purpose: This document contains the specification of a service robotics ontology for everyday tasks. The text is organized as follows: Section 1 presents the ontology main goal and applied domain and Section 2 describes the specification methodology. Section 3 lists the questions that guided the ontology specification, and Section 4 lists the knowledge data source. Section 5 presents the ontology dictionary of concepts, comprising each concept, its definition, and examples. In addition, the annexes contain a brief description of the set of concepts reused from other domain ontologies and upper ontologies.

1. Ontology Main Goal and domain

The main goal is to define a *well-defined knowledge base* for the service robot domain, which could:

- Prevent unambiguous communication.
- Support data integration and information exchange
- Support tools and framework implementation, e.g., tools for task planning and code generation.

2. Methodology

In order to define this ontology we integrate the development method proposed by IEEE (2015) and the guidelines proposed by (Vital and Café, 2011).

The work starts by defining the ontology's main goal and domain identification, as well as the specification of sources to acquire knowledge. Current ontologies should be considered due to its relevance for concepts reuse and specialization and to enable compatibility among this ontology and other ontologies.

The next step consists of defining the ontology's concepts and properties, and organizing them using an intermediate representation. In this project, we specify a *dictionary of concepts* in natural language and represent it in a metamodel.

Based on the dictionary of concepts, the ontology classes and hierarchy are specified as well as their axioms, object property and data property. Then, instances are defined for each class. In this project we used the Protégé tool for implementation.

3. Main Questions

- I. What are the parts of a robot?
- II. What are the capabilities of a robot?

- III. In which scenario can a service robot act?
- IV. What is a scenario composition in (sub-)areas?
- V. Who can control a robot?
- VI. What are the privileges of a given person?
- VII. What objects can a robot recognize?
- VIII. Where is an object in the environment?
- IX. What are the capabilities required to perform a task?

4. Reference ontologies

This project reuses / specializes concepts from current standards / ontologies. This is important to enable compatibility among specifications. The following ontologies are used as reference:

- Descriptive ontology for linguistic and cognitive engineering (DOLCE) ultra light (DUL) ;
- Suggested Upper Merged Ontology (SUMO) - is a top-level ontology that defines basic ontological categories across all domains;
- Core ontology for robotics and automation (CORA) – defines the main, most general concepts, relations, and axioms of robotics and automation;
- Autonomous Robotics (AuR) - extend the CORA ontology to represent more specific concepts and axioms that are commonly used in autonomous robotics.

5. Dictionary of Concepts

The concepts are organized into three categories:

- Concepts related to the robot's structure and behavior.
- Concepts related to the environment definition, where the robot will be inserted
- Concepts related to task definitions that can be performed when using the robot in an environment.

Through the dictionary, concepts are highlighted in bold. For each concept, the dictionary presents its definition and examples.

Reused/extended concepts from other ontologies bring the reference in parenthesis, e.g., (CORA) or (ISO8373:2021).

1.1 Concepts related to the robot structure and behavior

In general, a **ServiceRobot** is composed of various **ServiceRobotParts**, which include both software and hardware components. These parts work together to enable the robot's set of **Capabilities**, which are implemented through **ServiceRobotBehavior**.

ServiceRobot: is a **fully autonomous Robot** (from CORA) for personal or professional use that performs useful tasks for humans or equipment (ISO8373:2021). It may involve **human interaction** (ISO8373:2021) but does not allow **human intervention**.

Human Interaction (ISO8373:2021): “information and action exchanges between human and robot to perform a task by means of a user interface. Exchanges through vocal, visual and tactile means”.

Human intervention: external (remote or in-person) human control.

Personal use: includes tasks related to personal assistance such as cooking, cleaning, carrying things, and handling or serving of items.

Professional use includes tasks related to personal assistance in a non-structured environment, such as, serving dinner in a restaurant, welcoming a hotel guest.

A *ServiceRobot* has a name and a status, indicating whenever it is working, shutdown or charging.

* *ServiceRobot* does NOT include industrial robots, medical robots, or teleoperated robots. In some situations, a robot may assume more than one role, e.g., a hybrid *ServiceRobot* and teleoperated robot. Although this is possible, it is outside this ontology's scope.

* This ontology does not consider Robot Group (CORA). Cooperation among robots to achieve a goal may be considered in future releases.

ServiceRobotPart: represents the parts that compose a *ServiceRobot*. A *ServiceRobotPart* can be composed of another *ServiceRobotPart* (named subpart). A *ServiceRobot* may have hardware parts (*DevicePart*) or software parts (*SoftwarePart*). A part can be connected to other parts through the definition of *PartConnection*.

SoftwarePart: is an abstract entity (from SUMO) that represents the software components of a *ServiceRobot*, e.g., software functions, libraries, and applications.

DevicePart: represents the hardware parts (*RobotPart* in CORA) of a service robot, i.e. a *RobotProcessingPart* (from CORA), a *RobotSensingPart* (from CORA), a *RobotActuatingPart* (from CORA) or a *RobotCommunicatingPart* (from CORA). A *DevicePart* can be physically connected to the robot or logically connected (e.g. an external computer used during a task).

Examples of processing parts for the AtHome domain include computer processors and microcontrollers.

Examples of sensing parts for the AtHome domain include a camera, LIDAR, Kinect, and microphone.

Examples of actuating parts for the AtHome domain include arms, speakers, screens, and wheels.

Examples of communicating part for the AtHome domain include: hub.

PartConnection: is a connection (from SysML), physical or logical, between two *RobotPart*. For example, the connection of a *RobotSensingPart* (e.g. a camera) with a *RobotPart* (e.g. chassis), e.g. in Gazebo simulator called *Joint*; the connection of a *SoftwarePart* (e.g. a library) with a *RobotSensingPart* (e.g. camera), in Gazebo Simulator called *Plugin*.

Capability: is the combination of behaviors, robot parts, and their organization to deliver a specified outcome. A capability specifies WHAT the robot can deliver without defining HOW to do it.

ServiceRobotBehavior: represents a capability realization according to a specific implementation strategy. It orchestrates (plans) the set of *PrimitiveSkills*, necessary for a capability implementation. The orchestration consists of the coordination of *PrimitiveSkills*, i.e., involves the definition of a workflow or pipeline, with the *PrimitiveSkills* needed to perform the behavior. *PrimitiveSkills* can be orchestrated in parallel or sequentially using, for example, behavior tree or UML state machine.

PrimitiveSkill: consists of using service robot parts to produce a specific result. A *PrimitiveSkill* is a behavior ATOMIC STAGE, i.e., it CAN NOT be composed by other *PrimitiveSkill*.

Sensing: is a *PrimitiveSkill* related to sensing and capturing data from the environment. Examples are: see (capture a video) and hear (capture sounds).

Cognition: is a *PrimitiveSkill* related to data processing. Examples: Robot training and face recognition.

Actuating: is a *PrimitiveSkill* related to act and modify the environment. Examples: speak.

Contract: establishes an agreement between a *PrimitiveSkills* and a *ServiceRobotPart*. It links a specific *PrimitiveSkill* to the necessary parts for its implementation, i.e., there should be a *Contract* relating the *PrimiviteSkill* to the *ServiceRobotParts* required so that the *PrimitiveSkill* can be implemented/executed by a *ServiceRobot* that has that *ServiceRobotParts*.

A *PrimitiveSkill* may require n parts. For each part, a contract is defined. In a *Contract*, the *PrimitiveSkill* and the *ServiceRobotPart*'s non-functional requirements must be compatible* (*Match*). Examples: (i) a *SoftwarePart* that executes in 0.5s is compatible with a *PrimitiveSkill* that requires execution in a maximum of 1s; (ii) A camera (*RobotSensingPart*) that provides an image at 10 frames/s is compatible with a

PrimitiveSkill that requires an image at least 5 frames/s; (iii) a *DevicePart* that uses a maximum of 50mA is compatible with a *PrimitiveSkill* that requires up to 70mA.

Match: indicates the *PrimitiveSkill* requirement that must be met by the *ServiceRobotPart* requirement.

*The current version of the ServiceRobots ontology does not intend to verify non-functional requirements compatibility automatically, i.e., a human should do this work during behavior element specification.

Requirement: represents a proposition (from SUMO) that must be fulfilled. It has a description of its purpose, a type (reqType), belongs to a domain (reqDomain), and is specialized in *PartRequirement* and *CapabilityRequirement*.

RequirementTypeKind: indicates whenever it refers to a functional or non-functional requirement.

RequirementDomainKind: represents the requirement domain, e.g., usability, time (execution time), or distance (minimal distance, e.g., to see or hear).

7. Implementation

The ontology is implemented in Protegé tool and available at

Table 1 lists properties and axioms, according to the following legend:

Domain - domain name, applied to the property

Property - property name

Range - range name, applied to the property

F - Functional property

IF - Inverse Functional property

As - Asymmetric property

Ir - Irreflexive property

Opposite - name of the opposite property related to the property in col two

Table 1: Property details

Domain	Property	Range	F	IF	As	Ir	Opposit
ServiceRobot	hasComponent	ServiceRobotPart		x			isComponentOf
ServiceRobotPart	hasSubComponent	ServiceRobotPart		x	x	x	isSubcomponent

						Of
ServiceRobotPart	isParent	PartConnection				ConnectParent
ServiceRototPart	isChild	PartConnection				ConnectChild
ServiceRobot	Performs	Capability				isPerformedBy
Capability	isImplementedAs	ServiceRobotBeha vior		x		
ServiceRobotBeha vior	Coordinates	PrimitiveSkill				
Contract	involvesPart	ServiceRobotPart				isPartOfContract
Contract	involvesAction	PrimitiveSkill				isActionOfContra ct
Requirement	hasDomain	RequirementDom ainKind				
Requirement	hasType	RequirementDom ainType	x			
ServiceRobotPart	requiresReqPart	Requirement				
PrimitiveSkill	RequiresReqAction	Requirement				

References

- IEEE Standard Ontologies for Robotics and Automation. IEEE Std 1872™-2015
- IEEE Standard for Autonomous Robotics (AuR) Ontology. IEEE Std 1872.2™ - 2021.
- ISO 8373 Robotics Vocabulary. Available at <https://cdn.standards.iteh.ai/samples/75539/1bc8409322eb4922bf680e15901852d2/ISO-8373-2021.pdf> accessed 23/04/2024.
- SUMO, The Suggested Upper Merged Ontology. Available at www.ontologyportal.org accessed 23/04/2024.
- SysML profile. Available at <https://www.omg.sysml.org/SysML-2.htm> accessed 23/04/2024.
- Vital, Luciane; Café, Ligia. Ontologias e Taxonomias: diferenças. Perspectivas em Ciência da Informação, v.16, n.2, p.115-130, abr./jun. 2011

Modifications:

18/08 - change the concept terminology from PrimitiveAction to PrimitiveSkill